University of Exeter

ECM2433: The C Family Arrays & Strings

Richard Everson

Arrays

Arrays store many variables of the **same type** in **contiguous memory** that can be accessed by **indexing**

A 1-dimensional array of 5 characters, initialised with 5 values:

```
char array[5] = {'A' 'B', 'C', 'D', 'E'};
```

The size of the array is dictated by its initialisation, i.e. 10 ints

```
int  a[] = {9, 8, 7, 6, 5, 4, 3, 2, 1, 0};
```

It is guaranteed that arrays are stored in contiguous memory. This will become important later.

An uninitialised array:

```
int  uninitialised[10];     /* Contents are undefined */
```

An initialised 2-dimensional array:

```
int  a2[2][3] = { {1,2,3}, {4,5,6} };
```

These declarations reserve an appropriate amount of memory for each variable.

"int anArray[10];" reserves 10 x `sizeof(int)` = 10 x 4 bytes **(on this system)** = 40 bytes for `uninitialised`.

Arrays

University of Exeter

With these declarations:

```
char array[5] = {'A','B','C','D','E'};
int  a[]       = {9, 8, 7, 6, 5, 4, 3, 2, 1, 0};
int  uninitialised[10];
int  a2[2][3]  = { {1,2,3}, {4,5,6} };   /* Array of arrays */
```

Access and set individual values like this:

```
unitialised[0] = 42;
array[3]  = 'Z';
printf("%c   %d\n", array[4], a2[1][2]);
```

```
D   6
```

```
a2[0][2]   = 99;
```

Array bounds

```
#include <stdio.h>
int main()
{
    int N = 10, i;
    int  array[] = {9, 8, 7, 6, 5, 4, 3, 2, 1, 0};
    int  array2[N];
    for (i = 0; i < N; i++)
        array2[i] = i;

    array2[15] = 999;

    for (i = 0; i < N; i++)
        printf("%d ", array[i]);
    printf("\n");
    return 0;
}
```

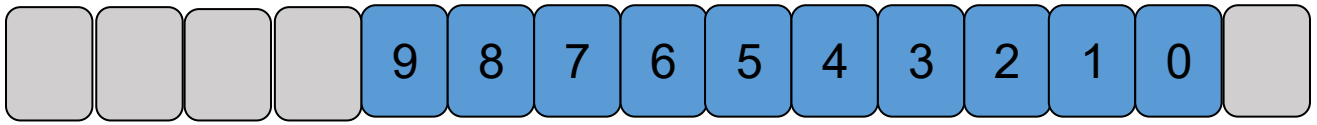
Bounds checking cannot be relied on

```
9 8 7 999 5 4 3 2 1 0
```

Precise results are machine dependent

Arrays in memory

```
int  array[] = {9, 8, 7, 6, 5, 4, 3, 2, 1, 0};
```

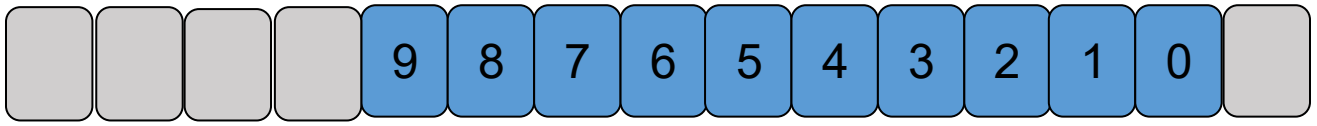


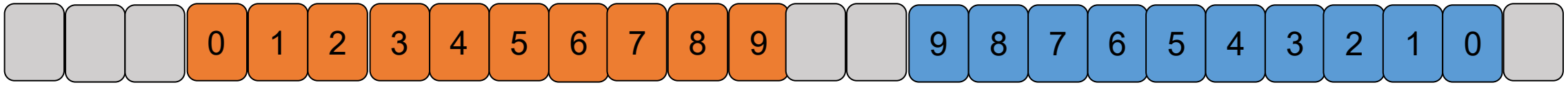
= 1 int word (= 4 bytes)

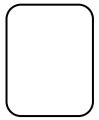
Arrays in memory

```
int  array[] = {9, 8, 7, 6, 5, 4, 3, 2, 1, 0};
```

```
int  array2[N];
for (i = 0; i < N; i++)
    array2[i] = i;
```

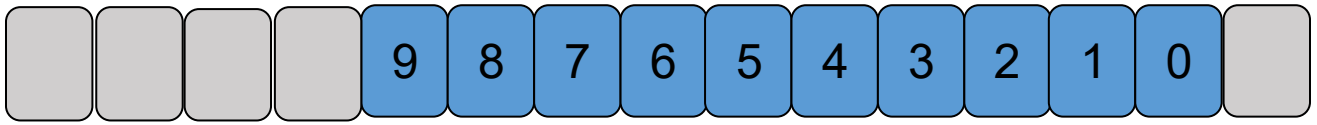




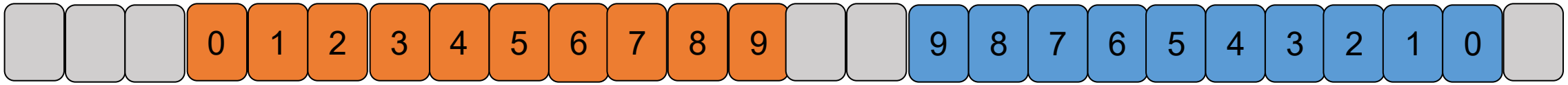

= 1 int word (= 4 bytes)

Arrays in memory

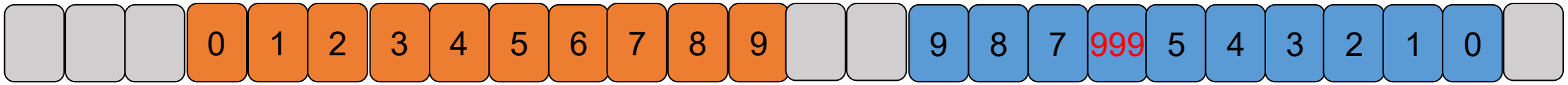
```
int  array[] = {9, 8, 7, 6, 5, 4, 3, 2, 1, 0};
```




```
int  array2[N];
for (i = 0; i < N; i++)
    array2[i] = i;
```



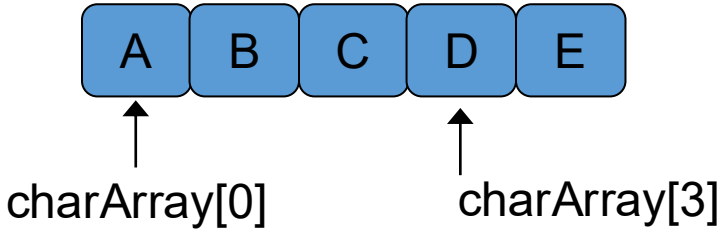

```
array2[15] = 999;
```



= 1 int word (= 4 bytes)

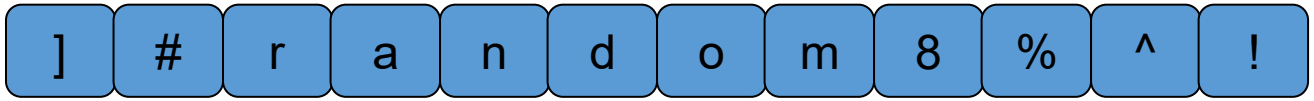
Arrays of chars

```
char charArray[5]   = {'A','B','C','D','E'};
```




```
char uninitialised[12];
```

`unitialised` is uninitialised, so contains random stuff

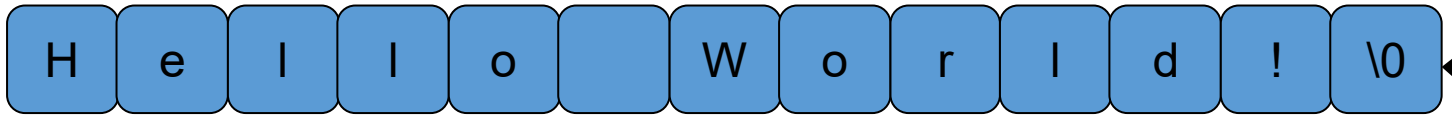


= 1 byte of memory

Strings are just arrays of chars with a null terminator

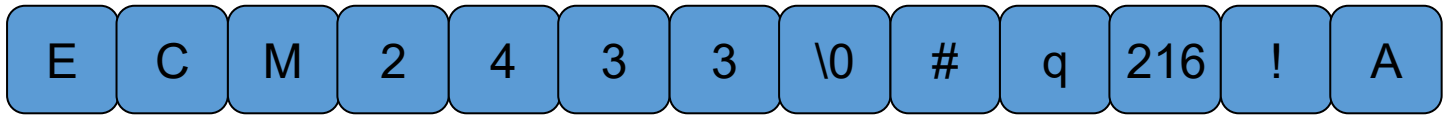
Note: a character uses single quotes, while a string uses double quotes

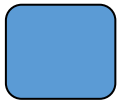
```
char s[] = "Hello World!";
```



the null character. All strings in C are "null terminated"

```
char name[13] = "ECM2433";
```





= 1 byte of memory

The null terminator

- Null terminator `'\0'` is the single byte with all bits 0.
- The null terminator ('\0') tells printf when to stop printing out characters and other functions when to stop processing the string.
- Memory allocated is number of characters plus one for the null terminator.

If your string does not have a null terminator, printf carries on plodding through memory and printing out characters until it encounters one.

```
char s[] = "Hello World!";
printf(s);
```

```
Hello World!
```

No newline!

```
printf("%s\n", s);
```

```
Hello World!Hello World!
```

The null terminator

```
char s[] = "Hello World!";

printf("%s\n", s);
```

```
Hello World!
```

```
s[4] = '\0';
printf("%s\n", s);
```

```
Hell
```

If your string does not have a null terminator, printf carries on plodding through memory and printing out characters until it encounters one.

The null terminator

```
char s[] = "Hello World!";

printf("%s\n", s);
```

```
Hello World!
```

```
s[4] = '\0';
printf("%s\n", s);
```

```
Hell
```

```
s[3] = 'p';
printf("%s\n", s);
```

```
Help
```

If your string does not have a null terminator, printf carries on plodding through memory and printing out characters until it encounters one.

The null terminator

```
char s[] = "Hello World!";

printf("%s\n", s);
```

```
Hello World!
```

```
s[4] = '\0';
printf("%s\n", s);
```

```
Hell
```

```
s[3] = 'p';
printf("%s\n", string);
```

```
Help
```

If your string does not have a null terminator, printf carries on plodding through memory and printing out characters until it encounters one.

```
s[4] = '\n';
printf("%s\n", string);
```

```
Help
 World!
```

Strings and printf formatting

University of Exeter

```
char name[21] = "Elephants";
printf("*%s*\n", name);
*Elephants*
printf("*%3s*\n", name);
*Elephants*
printf("*%20s*\n", name);
*            Elephants*
printf("*%-20s*\n", name);
*Elephants           *
printf("*%-20.3s*\n", name);
*Ele                 *
*Ele                 *
```

Strings and variable printf formatting

The printf width and precision may be defined at run-time:

```
int   wid  = 20;
int   prec = 10;
char  str[] = "My elephants";
printf("%*.*s\n", wid, prec, str);
```

is equivalent to

```
printf("%20.10s\n", "My elephants");
```

Special characters

character	meaning
\n	line-feed (new line)
\t	horizontal tab
\v	vertical tab
\b	backspace
\r	carriage return
\f	form-feed (new page)
\\	backslash
\'	single quote
\"	double quote
%%	percent sign

Note: the end of a line in Linux is just a line-feed; in Windows it is carriage return + line-feed

Common string functions

```
#include <string.h>
```

```
size_t strlen(char s[]);
```

returns the number of characters in `firstName`, not including the null terminator

```
strcmp(char s[], char t[]);
```

returns 0 if the two strings are the same; otherwise non-zero
Hence expressions like

```
if ( !strcmp(firstName,"Sam") ) {
     /* Strings are equal */
}
```

strcpy()

University of Exeter

Warning: Recipient strings/arrays must be large enough to contain the result!

```
strcpy(char dest[], char src[]);
```

Copies the string in `src` into `dest`, including the null terminator

strcpy()

University of Exeter

Warning: Recipient strings/arrays must be large enough to contain the result!

```
strcpy(char dest[], char src[]);
```

Copies the string in `src` into `dest`, including the null terminator

man strcpy says:

ADMONISHMENT

YOU SHOULD ALMOST CERTAINLY USE strlcpy() INSTEAD.

```
strlcpy(char dest[], char src[], size_t len);
```

Copies up to len-1 characters from src into dest, terminating with a NULL.

Common string functions

```
#include <string.h>
```

```
strlcat(char dest[], size_t len, char src[]);
```

Concatenates `src` onto the end of `dest`, overwriting `dest` from the first null terminator

```
snprintf(char buffer[], size_t len, char format[], args,. . )
```

Like `printf`, but places the result in `buffer` instead of printing out

```
char buf[128];
int age = 45;
char name[] = "Harry Potter";
snprintf(buf, sizeof(buf)-1, "% is %d\n", name, age);
printf(buf);
```

```
Harry Potter is 45
```

Summary

- Arrays
  - Arrays of contiguous memory
  - Bounds checking – cannot be relied on!

- Strings
  - A string is an array of characters
  - Terminated by the null character
  - printf for strings
  - Common string functions: `man string` etc